

Technical Report on the Implemented `main_2` Corrosion Pipeline

Repository-grounded reconstruction from code, data, artifacts, and local documentation

March 17, 2026

Abstract

This report reconstructs the `main_2` project strictly from the executable repository state, saved artifacts, generated diagnostics, the current raw workbook, and the local thesis and conference PDFs stored under `Documentation/`. The implemented system is a current-state corrosion regressor: it predicts the continuous surface label `B_Peak_Rust_Percentage_[%]` for a given image, optionally concatenated with experiment metadata, and exposes the selected model through a FastAPI endpoint for “current corrosion” prediction. It does not implement direct image-to-RUL learning, survival analysis, hidden structural damage estimation, or sequence forecasting.

The repository also contains important reproducibility drift. The saved artifacts document one successful run on 792 observations, but the current raw Excel workbook now has two header rows, 791 labeled data rows, no directly exposed `NaCl%` column, and a different identifier layout than the loader expects. The single corrupted image `E01-20240508-17W.png` is still present in the image directory and in the saved predictions, but its label row is absent from the current workbook. The report therefore treats the saved outputs as evidence of what was run, while explicitly marking what can no longer be rerun from the present raw files without code or data repair.

Contents

1	Problem framing	3
1.1	Actual task implemented in <code>main_2</code>	3
1.2	What is directly supervised and what is not	3
1.3	Why direct image-to-RUL is not supported	3
1.4	Exact scope of the implemented system	4
2	Repository-grounded pipeline overview	4
2.1	Key repository components	4
2.2	Data ingestion and dataset parsing	5
2.3	Image alignment and preprocessing status	5
2.4	Identifier mapping, image loading, and cleaning	6
2.5	Feature extraction and feature-family organization	6
2.6	Model training	6
2.7	Split strategy and evaluation	7
2.8	Degradation analysis and proxy-RUL logic	7
2.9	Deployment stage	7
3	Data description and data dictionary	8
3.1	Current workbook versus saved artifacts	8
3.2	Identifiers and metadata dictionary	8
3.3	Label and target dictionary	10

3.4	Label density and class balance	11
4	Feature engineering	12
4.1	Implemented feature families	12
4.2	What is not implemented in this repository version	12
4.3	Feature removal, exclusion, and downweighting	13
5	Technical challenges and implemented responses	13
6	Modeling methodology	15
6.1	Target, preprocessing, and optimization	15
6.2	Image-only branch	15
6.3	Multimodal branch	15
6.4	Exact saved training outcomes	16
7	Split strategies and leakage control	16
7.1	Why random row-wise splitting would be invalid	16
7.2	Implemented grouped splitting	16
7.3	What the implemented split supports scientifically	17
7.4	What is absent	17
8	Results and interpretation	17
8.1	Diagnostic figures from the saved run	17
8.2	Strongly supported results	19
8.3	Weak, negative, and inconclusive results	20
8.4	What the results do <i>not</i> justify claiming	21
9	What the model can and cannot learn	21
9.1	What is directly visible and therefore learnable	21
9.2	What may be indirectly inferable	21
9.3	What is hidden and likely not reliably recoverable here	21
9.4	Why metadata can help but also mislead	22
9.5	Scientific implication	22
10	Traceability to the original thesis and documentation	22
10.1	Documented method versus executable repository	22
10.2	What could not be recovered	23
10.3	How the report handles these mismatches	23
11	Limitations	23
12	Future work	24
A	Meeting-defense questions and concise evidence-based answers	25

1 Problem framing

1.1 Actual task implemented in main_2

The executable training pipeline in `main_2/train_phase2.py` solves a single supervised regression problem: predict the corrosion label `B_Peak_Rust_Percentage_[%]` for each image-timepoint pair. The selected saved artifact is an image-only multilayer perceptron (MLP) fed by a frozen 512-dimensional ResNet-18 embedding. A second model, named “multimodal”, concatenates the same image embedding with tabular metadata, but this branch was not selected for deployment in the saved run.

This is not a future forecasting pipeline. The model input image and the target belong to the same observation row, and no target shifting, sequence windowing, autoregressive modeling, or future-week prediction logic appears in the code. The generated “progression” plots in `main_2/visualize.py` are retrospective diagnostics over already observed weeks; they are not forecast trajectories.

1.2 What is directly supervised and what is not

The only direct supervised target in `main_2/config.py` is:

- `B_Peak_Rust_Percentage_[%]`: a continuous corrosion quantity defined in the workbook and treated as the regression target.

The following quantities exist in the current workbook but are not used as learning targets in `main_2`:

- `A_Surface_Total_Rust_Percentage_[%]`
- `A_Total_Rust_Category_(1-4)`
- `B_Peak_Rust_Category_(1-4)`
- `B_Location_of_Peak_Rus_ in_length_[cm]`
- `Last_Wire_Area_Loss_(Faliure_Surface)_%`
- `Ultimate_Load_[kN]`

The structural variables are especially sparse: in the current workbook they appear only once per specimen (48 non-null values each over 791 labeled rows), which makes them endpoint-type measurements rather than dense frame-level supervision. No code in `main_2` trains against them, regresses them, thresholds them, or converts them into remaining useful life.

1.3 Why direct image-to-RUL is not supported

Direct image-to-RUL is unsupported for three repository-grounded reasons:

1. No RUL label exists in the executable pipeline. Neither the workbook nor the artifacts contain a dedicated remaining-life column.
2. No proxy-RUL logic exists in `main_2`. There is no threshold-crossing model, no survival/event-time formulation, no degradation fit, and no time-to-failure loss.
3. The served API endpoint is explicitly `/predict/current`, and the inference function is named `predict_current_corrosion`; it clips a current corrosion percentage to `[0, 100]` and maps it to low/medium/high severity bins.

1.4 Exact scope of the implemented system

The implemented scope of `main_2` is:

- read an Excel table and derive per-row metadata from an identifier string;
- assign train/validation/test splits grouped by specimen;
- extract frozen ResNet-18 embeddings from PNG images;
- optionally concatenate tabular metadata after imputation, scaling, and one-hot encoding;
- train two regression MLPs for current peak rust;
- compare those models on a held-out grouped test split;
- save one selected model, prediction CSVs, and diagnostic figures;
- expose the selected current-corrosion model through a small FastAPI service.

Everything beyond that scope, including structural degradation prediction, campaign-transfer benchmarking, or RUL estimation, would be an extrapolation beyond what the code actually does.

2 Repository-grounded pipeline overview

2.1 Key repository components

Component	Repository-grounded role
<code>main_2/config.py</code>	Central constants: repository paths, target column, base tabular feature list, split fractions, and random seed.
<code>main_2/data.py</code>	Data loading, identifier parsing, grouped split generation, tabular preprocessing, ResNet-18 embedding extraction, corrupted-image fallback, split labeling, and dataset summary.
<code>main_2/models.py</code>	Single regression architecture: three-layer MLP with batch normalization and dropout.
<code>main_2/train_phase2.py</code>	End-to-end training script: load data, build grouped splits, extract embeddings, train image-only and multimodal models, compare on test, save the selected artifact, write CSV outputs, and call visualization/report helpers.
<code>main_2/visualize.py</code>	Generates learning diagnostics, parity plots, residual plots, group-wise MAE tables, severity confusion matrix, and per-specimen trajectory figures.
<code>main_2/report_writer.py</code>	Writes the generated markdown report. Its prose is broader than the actual saved artifact and is therefore not taken as primary evidence where it conflicts with code or outputs.
<code>main_2/build_pdf_report.py</code>	Builds a PDF-style narrative report from already saved outputs. It contains useful artifact summaries but also discusses candidate and future methods that are not implemented in <code>main_2</code> .
<code>main_2/inference.py</code> and <code>main_2/api.py</code>	Load the saved artifact and expose current corrosion inference via FastAPI. In the repository's current state, loading fails because <code>run_info.json</code> stores stale absolute paths from another machine.
<code>main_2/reports/</code> and <code>main_2/artifacts/</code>	Primary evidence of the saved run: metrics, training history, predictions, figures, model state, preprocessor, scaler, and metadata.

Table 1: Executable components used to reconstruct the actual `main_2` pipeline.

2.2 Data ingestion and dataset parsing

Input. The intended input is `Data/Images_Dataset_A-Z.xlsx` together with PNG images in `Data/Images_dataset/`. The saved run metadata in `main_2/artifacts/run_info.json` records 792 rows, 48 specimens, weeks 0–36, and one corrupted image.

Implemented parsing logic. `main_2/data.py` reads the Excel file with `pd.read_excel(excel_path)`, then assumes the first spreadsheet row contains the semantic headers. It converts every column except ID and Treatment to numeric, and it parses ID using the regular expression

$$\sim(?P<specimen>[A-Za-z0-9]+)-(?P<date>\d{8})-(?P<week>\d+)W\$.$$

From this, it derives:

- specimen
- capture_date
- week
- series
- image_path = image_dir / (ID + ".png")

Current repository mismatch. The current raw workbook no longer matches that contract. It has two header rows, not one; the full row identifier now appears in `Sample Name`, while ID stores only specimen-level IDs such as D01; and the workbook exposes `Ageing Days`, not `Ageing_Days`, and no explicit `NaCl%` column. As a consequence, the current `load_dataframe()` implementation no longer reproduces the saved run from the current workbook.

Purpose. This stage exists to align every label row with one image file and to derive the grouping variable (`specimen`) and temporal variable (`week`) required for split construction and diagnostics.

Risks and limitations. The parsing stage is brittle because it hard-codes a workbook schema and a specific identifier layout. Once the workbook changed, the training code became non-reproducible even though the saved artifacts remained on disk.

2.3 Image alignment and preprocessing status

Implemented behavior. No explicit image alignment, registration, cropping, white-balance correction, mask generation, or geometric normalization is implemented in `main_2`. The only executable image preprocessing is the standard transform returned by `ResNet18_Weights.IMAGENET1K_V1.transform` applied immediately before embedding extraction.

Purpose. This keeps the executable pipeline small: the model consumes RGB PNGs directly and relies on transfer-learned ResNet features instead of a custom corrosion segmentation workflow.

Limitation. Because no repository-executed alignment or white-balance stage exists here, `main_2` does not reproduce the GIMP/BIMP/MATLAB image preparation documented in the thesis and conference paper. That mismatch matters because visible rust detection is sensitive to illumination, cropping, and color normalization.

2.4 Identifier mapping, image loading, and cleaning

Input. Each row’s identifier is converted into a resolved PNG path. The code then iterates through the dataframe and tries to open each image with Pillow.

Output. For each row, the image pipeline outputs either a 512-dimensional embedding or an all-NaN placeholder, later replaced by per-dimension medians.

Implemented handling of bad inputs. The image `E01-20240508-17W.png` is unreadable by Pillow (`UnidentifiedImageError`). The saved run records this ID in `corrupted_image_ids`. The corresponding embedding is replaced by column medians after all embeddings are stacked.

Residual limitation. This fallback preserves array shape and allows training to continue, but it replaces image information with a generic feature vector. The corrupted row is also the one row present in the 792-image directory and saved predictions but absent from the current 791-row workbook, so it now contributes to a raw-data/artifact mismatch as well.

2.5 Feature extraction and feature-family organization

Image branch. `extract_resnet18_embeddings()` loads a pretrained torchvision ResNet-18 with `IMAGENET1K_V1` weights, replaces the final fully connected layer with `Identity()`, and stores the resulting 512-dimensional vector for each image. The backbone is used in evaluation mode and is not fine-tuned.

Tabular branch. `fit_tabular_preprocessor()` splits the configured feature list into numeric and categorical columns. Numeric columns receive median imputation plus standardization; categorical columns receive most-frequent imputation plus one-hot encoding with `handle_unknown="ignore"`.

Feature-family organization actually present. The code therefore has only two implemented feature families:

1. deep image embeddings;
2. preprocessed metadata/context features.

There is no handcrafted corrosion-feature block in this repository version.

2.6 Model training

Candidate models. Two models are trained:

- **image-only MLP:** input is the 512-dimensional image embedding;
- **multimodal MLP:** input is the concatenation of image embedding and preprocessed tabular features.

Shared training protocol. Both models use the same `RegressionMLP` architecture, standardized targets, `SmoothL1Loss`, `AdamW`, early stopping on validation MAE, batch size 64, learning rate 10^{-3} , weight decay 10^{-4} , dropout 0.2, and patience 18.

Purpose. The image-only branch tests whether visible corrosion signal is sufficient. The multimodal branch tests whether experiment metadata improves prediction beyond image evidence.

Limitation. There is no hyperparameter search, no classical baseline, no uncertainty model, and no temporal model in the executable code.

2.7 Split strategy and evaluation

Implemented split logic. `build_group_splits()` performs a nested `GroupShuffleSplit` by `specimen`: first 20% of specimen groups are assigned to test, then 20% of the remaining specimen groups are assigned to validation.

Outputs. The saved prediction file contains 504 train rows, 129 validation rows, and 159 test rows, corresponding to 30, 8, and 10 unique specimens respectively.

Metrics. The script computes MAE, RMSE, and R^2 on the test split. `visualize.py` then generates by-treatment, by-series, and by-specimen error summaries, but those group summaries are computed only on the test subset.

Methodological weakness. The code chooses the “best” model by sorting `model_comparison.csv` on `test_mae`. This means the test set is used not only for final reporting but also for model selection, which weakens the defensibility of the reported test estimate.

2.8 Degradation analysis and proxy-RUL logic

No degradation analysis or proxy-RUL stage is implemented in `main_2`. There is no degradation curve fitting, no threshold-crossing analysis, no event-time formulation, and no downstream transformation from current corrosion predictions to structural life. If the broader project goal includes those stages, they are not recoverable from the `main_2` executable pipeline.

2.9 Deployment stage

Implemented behavior. `inference.py` reloads the saved regression model, reproduces the ResNet-18 embedding step for one image, optionally concatenates tabular inputs if `use_tabular` is true, predicts a corrosion percentage, clips it to $[0, 100]$, and maps it into three severity bins: low $< 1\%$, medium $1\% - < 20\%$, and high $\geq 20\%$.

Current limitation. In the repository’s present state, `load_phase2_artifacts()` fails because `run_info.json` stores absolute artifact paths on another machine (`/Users/youseffayyaz/...`). The local artifact files exist, but the metadata points elsewhere, so the FastAPI service is not currently reproducible without repair.

3 Data description and data dictionary

3.1 Current workbook versus saved artifacts

Source	Rows	Specimens	Repository-grounded note
Current raw workbook Data/Images_Dataset_A-Z.xlsx	793 total	48	The workbook currently has two header rows and 791 labeled data rows.
Current labeled workbook rows after removing both headers	791	48	The row E01-20240508-17W is absent from the current workbook.
Current image directory Data/Images_dataset/	792 PNGs	48	Includes the corrupted file E01-20240508-17W.png.
Saved run_info.json and predictions_best_model.csv	792	48	Evidence of an earlier successful run with 792 supervised rows and one corrupted image handled by embedding imputation.

Table 2: The current raw data and the saved run artifacts no longer match one-to-one.

3.2 Identifiers and metadata dictionary

Variable	Meaning	Role	Repository-grounded notes
Sample Name	Full row identifier of the form specimen-date-weekW in the current workbook.	Original identifier in current raw file.	Present in the current workbook but not used by the current loader, even though it now contains the information the regex parser needs.
ID	Specimen identifier in the current workbook; examples include D01 and S2SA07.	Original metadata in current raw file.	The current loader wrongly assumes this column still contains full row-level IDs. That assumption breaks parsing.
specimen	Parsed specimen/material identifier.	Derived grouping variable.	In the saved run it is derived from the full row ID. In the current workbook it can be recovered from Sample Name , not from the current ID.
capture_date	Parsed calendar date embedded in the row-level image ID.	Derived temporal metadata.	Implemented in code but not recoverable from the current ID column without adapting the loader.

Variable	Meaning	Role	Repository-grounded notes
week	Parsed ageing week index.	Derived temporal feature and diagnostic axis.	Used as a tabular feature and in progression plots. Parsed from the identifier string rather than from a dedicated workbook column.
series	Alphabetic family extracted from the specimen prefix.	Derived meta-data feature.	Series values in the saved artifacts are {D, E, F, G, S}. This feature may encode campaign family rather than corrosion alone.
N_Steel_Mesh	Number of steel mesh layers.	Original tabular feature.	Used by the multimodal branch. In the current workbook it is present and numeric.
Treatment	Treatment code such as MI, NO, PA, SA, SA_PA, SA_VF, VF.	Original categorical feature.	Used by the multimodal branch and diagnostic group summaries.
Label_Treatment	Short label group in the current workbook.	Original meta-data, unused in <code>main_2</code> .	Values in the current workbook are {D, G, MI, SA, VF}. The repository does not document a full semantic mapping, so it remains ambiguous.
Ageing_Days	Exposure duration in days in the current workbook.	Original meta-data, not directly consumed by the current code.	The code expects Ageing_Days with an underscore, so the current workbook name does not match the loader contract.
Ageing_Days	Exposure duration in days expected by the saved pipeline.	Intended numeric tabular feature.	Recorded in BASE_FEATURES but not directly present under that name in the current workbook.
Cover_(Failure_Surface_Thickness)	Coil thickness on the failure surface, in millimeters.	Original numeric tabular feature.	Used by the multimodal branch and available in the current workbook.

Variable	Meaning	Role	Repository-grounded notes
NaCl%	Salt concentration feature expected by the multimodal branch.	Intended numeric tabular feature.	Present in the saved feature list and in stale generated material-catalog tables, but not directly exposed as a current workbook column. Its current provenance cannot be reconstructed from the raw workbook alone.

Table 3: Identifier and metadata fields relevant to `main_2`.

3.3 Label and target dictionary

Variable	Meaning and unit	Density	Role	Notes and ambiguities
A_Surface_Total_Rust_Percentage[%]	Total surface rust percentage over the analyzed surface.	791/791	Original label, unused target.	Dense across the current workbook. Mean is 2.85 and maximum is 53.412.
A_Total_Rust_Category_(1-4)	Original corrosion category associated with surface rust.	791/791	Original label, unused target.	Category thresholds are not documented in the repository.
B_Peak_Rust_Percentage[%]	Peak local rust percentage.	791/791 in current workbook; 792/792 in saved predictions	Primary super-vised target.	This is the only target actually used by <code>main_2</code> . Mean is 6.53 and maximum is 80.172 in the current workbook.
B_Peak_Rust_Category_(1-4)	Original category linked to peak rust.	791/791	Original label, unused target.	The repository does not define the numerical cutoffs for categories 1–4.
B_Location_of_Peak_Rust_in_length_[cm]	Location of the peak rust along specimen length, in centimeters.	791/791	Original label, unused target.	Not used in training or diagnostics in <code>main_2</code> .

Variable	Meaning and unit	Density	Role	Notes and ambiguities
Last_Wire_Area_Loss_(Failure_Surface)_%	Failure surface area loss on the failure surface.	48/791	Structural endpoint, unused.	One non-null value per specimen. Numeric values lie in $[0, 0.5698]$ despite the % suffix, so whether this is stored as percentage points or fraction is unresolved from repository evidence alone.
Ultimate_Load_[kN]	Ultimate load in kilonewtons.	48/791	Structural endpoint, unused.	One non-null value per specimen. Numeric range is 1.60–2.87 kN in the current workbook.

Table 4: Label fields visible in the current workbook and their actual role in `main_2`.

3.4 Label density and class balance

Quantity	Observed values	Use in main_2	Interpretation
A_Total_Rust_Category_(1-4)	1: 658, 2: 99, 3: 20, 4: 14	Not used	Strong low-category dominance.
B_Peak_Rust_Category_(1-4)	1: 526, 2: 107, 3: 95, 4: 63	Not used	Still imbalanced, but less extreme than the surface-rust category.
Structural endpoints (Last_Wire_Area_Loss, Ultimate_Load)	48 non-null each	Not used	One terminal label per specimen only; unsuitable for frame-level supervision without a different modeling design.
Three-level post-hoc severity bins used in diagnostics	low $< 1\%$, medium $1\% - < 20\%$, high $\geq 20\%$	Derived from B_Peak_Rust	Implemented only in <code>visualize.py</code> and <code>inference.py</code> ; not a native workbook label.

Table 5: Observed imbalance and sparsity patterns that shape what can be learned.

4 Feature engineering

4.1 Implemented feature families

Feature family	How it is computed	Physical interpretation and likely correlation	Weaknesses and risks
Frozen ResNet-18 embeddings (512-d)	RGB PNG passed through torchvision ResNet-18 (IMAGENET1K_V1) with final FC layer removed.	Encodes broad visual cues such as rust color, stain extent, texture, crack-like structures, and global appearance. Because the target itself is a visible surface-rust quantity, these embeddings can correlate strongly with current peak rust.	Low interpretability; no explicit localization; sensitive to capture conditions and preprocessing mismatch; may absorb campaign-specific imaging signatures. Prediction of a visible corrosion label from the same surface also has partial reconstructive/tautological character.
Numeric metadata	Median imputation and standardization of <code>N_Steel_Mesh</code> , <code>NaCl%</code> , <code>Ageing_Days</code> , <code>Cover_(Failure_Surface)</code> , and <code>week</code> .	Encodes specimen design, exposure duration, saline regime, and cover geometry. These may correlate with corrosion because the experiment itself is structured by treatment and time.	Shortcut-learning risk is high: <code>week</code> , <code>NaCl%</code> , and structural family can act as proxies for expected corrosion level rather than image evidence. In the current raw workbook, <code>NaCl%</code> and <code>Ageing_Days</code> are not directly reproducible under the names the code expects.
Categorical metadata	Most-frequent imputation plus one-hot encoding of <code>Treatment</code> and <code>series</code> .	Encodes protective-treatment family and specimen campaign.	High confounding risk. These features can encode group identity, and their value under stricter campaign transfer was not tested.
Post-hoc severity bins	Thresholding of continuous predictions into low/medium/high at 1% and 20%.	Provides an easier-to-read summary of regression behavior.	Not used for training; hides magnitude errors inside broad bins; class thresholds are hard-coded in code rather than inferred from the workbook.

Table 6: Actual feature families implemented in `main_2`.

4.2 What is not implemented in this repository version

The following feature families are *not* implemented anywhere in the executable `main_2` pipeline:

- rust masks or binary segmentation maps;
- RGB threshold features;

- color histograms;
- handcrafted texture descriptors;
- morphology measurements;
- strip-wise corrosion summaries;
- explicit rust-area ratios or normalized counts;
- manually engineered image descriptors intended to reproduce the thesis or conference workflow.

This matters because the thesis and conference documentation describe GIMP/MATLAB RGB-threshold pipelines and strip-based quantification. Those steps are not recoverable from the `main_2` codebase itself.

4.3 Feature removal, exclusion, and downweighting

There is no explicit feature-selection, removal, or downweighting routine in `main_2`. The relevant exclusion decisions are indirect:

- the selected saved artifact excludes all tabular inputs because the image-only model had the better *test* MAE in the saved run;
- corrupted-image embeddings are replaced by feature-wise medians;
- the API clips final outputs to $[0, 100]$, but the offline prediction CSV does not clip them.

As a result, the deployed artifact is image-only, but the repository still saves a tabular preprocessor and still documents a multimodal alternative. That should not be misread as evidence that tabular fusion was the final deployed solution.

5 Technical challenges and implemented responses

Challenge	How it was detected	Implemented handling	Residual risk
Workbook schema drift	The current workbook has two header rows, row-level IDs in <code>Sample Name</code> , <code>Ageing Days</code> instead of <code>Ageing_Days</code> , and no explicit <code>NaCl%</code> column. The current loader no longer reproduces the saved run.	None inside <code>main_2</code> . The report must rely on saved artifacts instead of a fresh rerun.	Reproducibility is broken until the loader and the data contract are repaired.

Challenge	How it was detected	Implemented handling	Residual risk
Corrupted image file	Pillow cannot identify E01-20240508-17W.png. The saved run records it in <code>corrupted_image_ids</code> .	Replace its 512-d embedding by column medians after stacking all embeddings.	The row keeps a label and prediction in the saved artifacts but now lacks a matching current workbook row; image information for that observation is lost.
Repeated-measures leakage risk	Each specimen appears over many weeks; random row-wise splitting would mix near-duplicate material states across train and test.	Nested <code>GroupShuffleSplit</code> by specimen.	Leakage by exact specimen identity is controlled, but distributional overlap by treatment and campaign remains.
Split imbalance	The saved test split has no E-series and no VF treatment; the validation split has no G-series.	None beyond grouped splitting.	Generalization claims are limited for missing or weakly represented groups.
Metadata shortcut learning risk	The multimodal model uses <code>week</code> , <code>series</code> , <code>Treatment</code> , and intended saline information. Multimodal validation MAE is better than image-only, but test MAE is worse.	Compare image-only versus multimodal variants.	The code does not quantify metadata contribution cleanly and does not test leave-one-group-out robustness.
Test-set model selection	<code>train_phase2.py</code> chooses the best model by sorting on <code>test_mae</code> .	None.	Reported test performance is not a strictly untouched final estimate.
Unbounded regression outputs	<code>predictions_best_model</code> contains negative values, including 13 negative predictions on the test split.	Only the online inference path clips predictions to $[0, 100]$.	Offline metrics and figures are computed on unclipped values, while API outputs have different semantics.
Artifact path portability	<code>run_info.json</code> stores absolute paths under another user's home directory. Loading the API artifact fails locally.	None in repository state.	Deployment cannot be reproduced locally without editing metadata or code.

Table 7: Real implementation and reproducibility problems found in the current repository state.

6 Modeling methodology

6.1 Target, preprocessing, and optimization

Both candidate models regress the same target, `B_Peak_Rust_Percentage_[%]`. The target is standardized with `StandardScaler` before training, and predictions are inverse-transformed before metric computation. The regressor architecture in `main_2/models.py` is fixed:

- input $\rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 1$;
- ReLU activations;
- batch normalization after the first two hidden layers;
- dropout after the first two hidden layers.

The optimizer is `AdamW` with learning rate 10^{-3} and weight decay 10^{-4} . The loss is `SmoothL1Loss`. The maximum number of epochs is 120, and early stopping patience is 18 epochs without validation-MAE improvement.

6.2 Image-only branch

Feature set. Only the 512-dimensional ResNet-18 embedding is used.

What this branch can learn. It can learn a direct mapping from visible surface appearance to the same-row peak-rust label. Because the selected saved model is this branch, the main reported performance in `main_2` is image-driven rather than metadata-driven.

What this branch cannot learn. It cannot directly learn hidden internal wire loss or residual structural capacity because those labels are not supervised in `main_2`. It also cannot forecast future corrosion because it does not consume sequences or future targets.

6.3 Multimodal branch

Feature set. The multimodal branch concatenates the image embedding with preprocessed metadata from `BASE_FEATURES`: `N_Steel_Mesh`, `Treatment`, `NaCl%`, `Ageing_Days`, `Cover_(Faliure_Surface)_[mm]`, `week`, and `series`.

Selection logic. The code trains this model in parallel with the image-only branch and saves the one with the smaller test MAE.

Interpretation. In the saved run, the multimodal model achieved a lower best validation MAE than the image-only branch but a worse test MAE. That combination is consistent with metadata carrying useful signal while also increasing sensitivity to split composition or group-specific structure.

6.4 Exact saved training outcomes

Model	Inputs	Epochs run	Best MAE	val	Test MAE	Test RMSE	Test R^2
image_only_mlp	512-d image embedding	32	4.9595		5.6255	9.2639	0.5237
multimodal_mlp	image embedding + metadata	63	4.0161		5.7476	9.8652	0.4599

Table 8: Saved model comparison reconstructed from `training_history.csv` and `model_comparison.csv`. The image-only branch was selected because the code sorts on `test_mae`.

The final selected artifact recorded in `run_info.json` is:

- `best_model = image_only_mlp`
- `input_dim = 512`
- `use_tabular = false`

Two further methodological caveats follow directly from the code:

1. the final choice is based on the test split rather than validation only;
2. the selected model is not retrained on train+validation after model choice. The saved state corresponds to the checkpoint found during the original train/validation process.

7 Split strategies and leakage control

7.1 Why random row-wise splitting would be invalid

Each specimen is observed repeatedly over time, usually for 15 or 18 weekly images. A random row-wise split would place highly related views of the same physical specimen into both training and testing. Because corrosion progression within a specimen is gradual and the imaging setup is likely consistent within a specimen, that would inflate apparent generalization.

7.2 Implemented grouped splitting

The current code avoids direct repeated-measures leakage by grouping on `specimen`. The saved split composition reconstructed from `predictions_best_model.csv` is shown in Table 9.

Split	Rows	Specimens	Series present		Treatment coverage and notes
Train	504	30	D:72, F:72, S:180	E:90, G:90,	MI, NO, PA, SA, SA_PA, SA_VF, VF all present.
Validation	129	8	D:18, F:18, S:75	E:18,	NO, PA, SA, VF present; G-series and several treatments absent.
Test	159	10	D:18, G:18, S:105	F:18,	MI, NO, PA, SA, SA_PA, SA_VF present; E-series and VF treatment absent.

Table 9: Actual grouped split composition reconstructed from the saved prediction file.

7.3 What the implemented split supports scientifically

The grouped split supports only one narrow conclusion: the saved model can generalize from some specimens to unseen specimens *within the same broad experimental repository*, under a split that still shares many treatment and campaign characteristics between train and test.

It does *not* support any strong conclusion about:

- leave-one-treatment-out generalization;
- leave-one-campaign-out generalization;
- temporal extrapolation to unseen future weeks;
- transfer to new image preprocessing conditions;
- inference of structural failure state.

7.4 What is absent

No leave-one-treatment-out, leave-one-series-out, leave-one-campaign-out, or time-forward split is implemented in `main_2`. Therefore any claim that stricter split regimes were evaluated, or that their performance degraded by a specific amount, would be unsupported in this repository version. The most that can be said is that the observed multimodal validation-test inversion and the strong group heterogeneity suggest sensitivity to split composition.

8 Results and interpretation

8.1 Diagnostic figures from the saved run

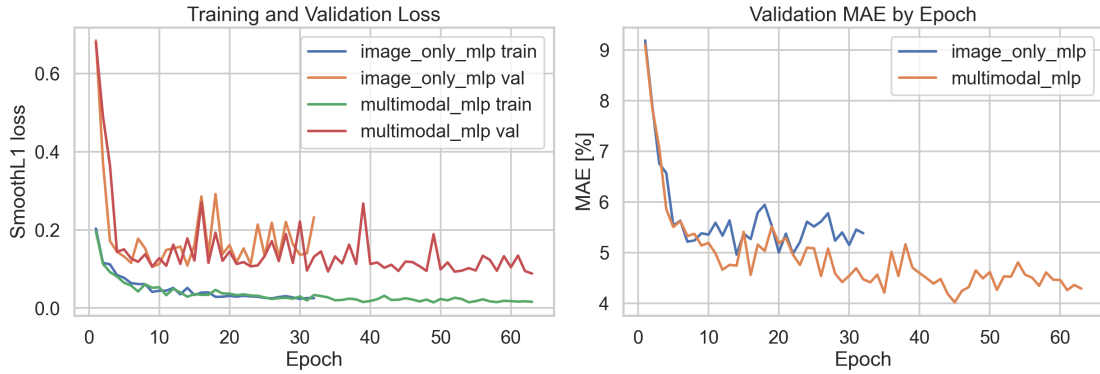


Figure 1: Training and validation curves for the two regression models. The multimodal branch reaches the lower validation MAE, but that did not translate into the better held-out test MAE.

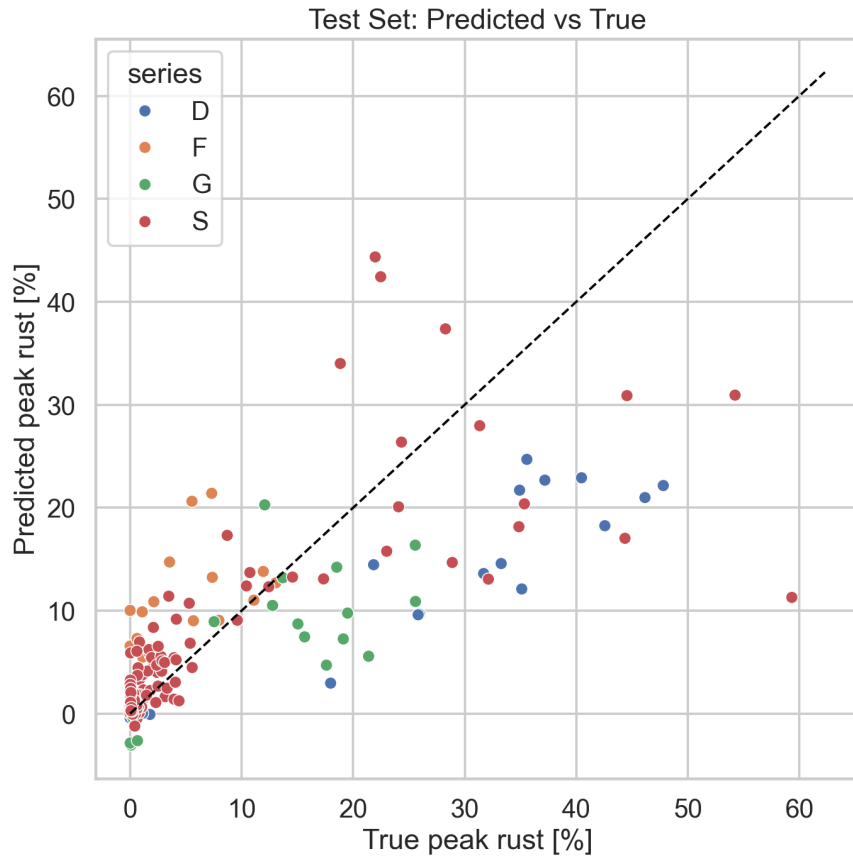


Figure 2: Predicted versus true peak rust on the grouped test split. The spread widens at higher rust values, and severe cases are often underpredicted.

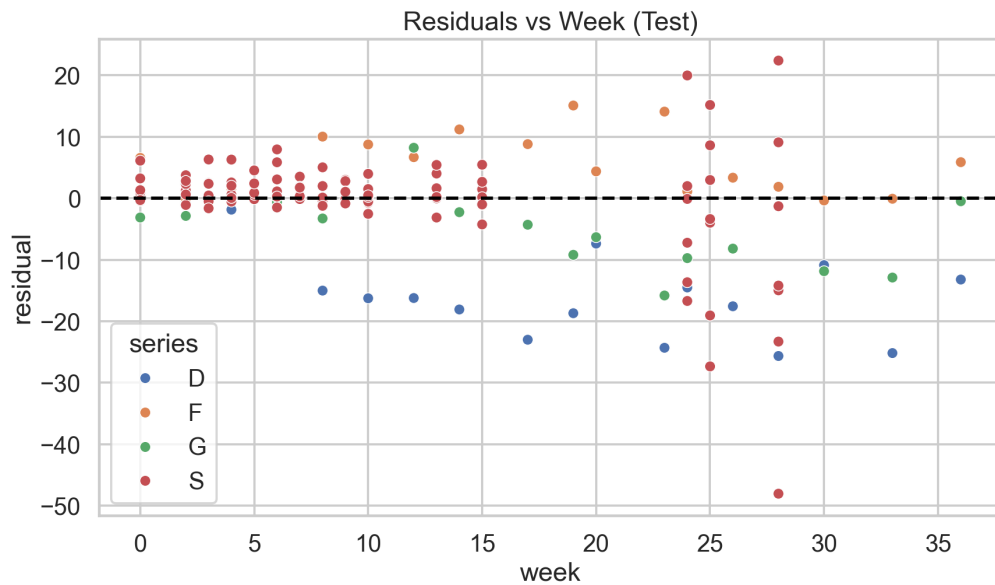


Figure 3: Residuals versus week on the grouped test split. Errors are small near week 0 and become increasingly negative at later weeks, indicating underprediction of severe late-stage corrosion.

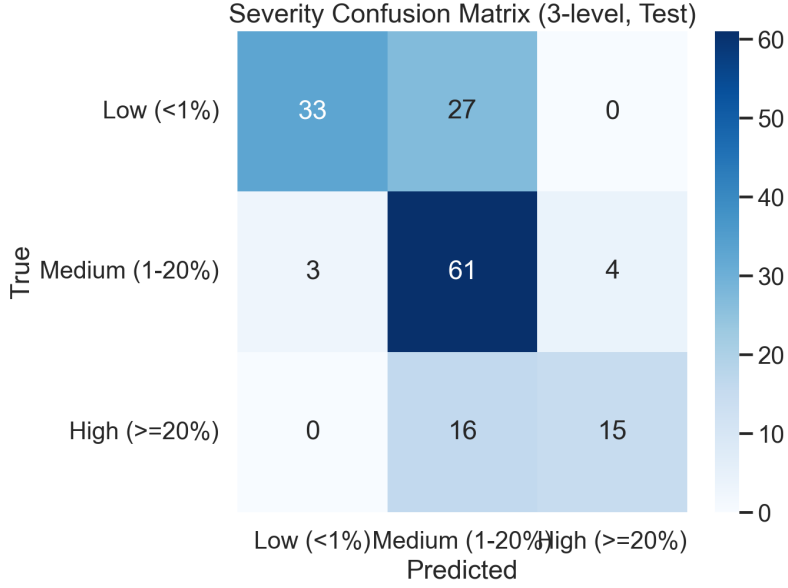


Figure 4: Three-level severity confusion matrix generated from the regression output using thresholds $< 1\%$, $1\% - < 20\%$, and $\geq 20\%$. The model is strongest in the medium band and weaker at both extremes.

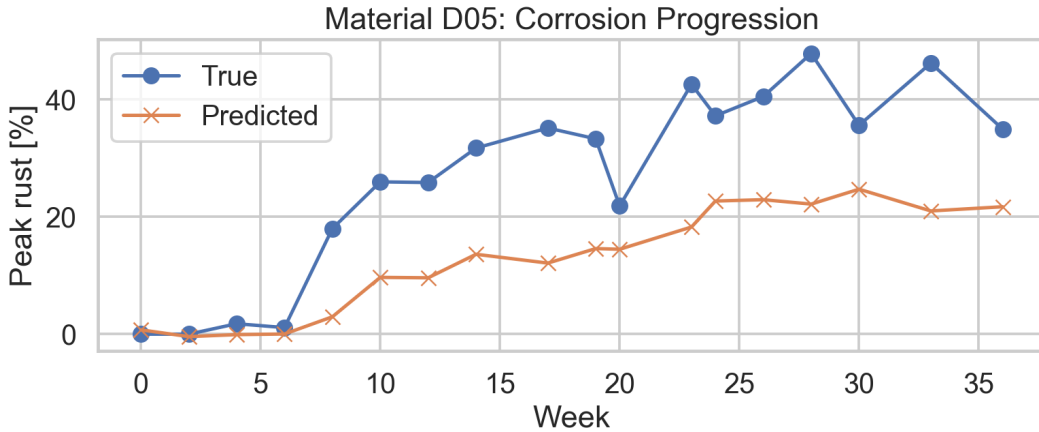


Figure 5: Per-specimen trajectory diagnostic for test specimen D05. This plot compares fitted values with already observed weekly labels; it is not a forecast trajectory. The model systematically underestimates the severe late-stage rise for this specimen.

8.2 Strongly supported results

1. The selected model learns current visible corrosion reasonably well under grouped specimen holdout. The saved image-only model achieves test MAE 5.6255, test RMSE 9.2639, and $R^2 = 0.5237$ on the grouped test split. This is the strongest technically supported performance statement in `main_2`.

2. The dominant saved signal is image-driven, not multimodal. The final selected artifact uses only image embeddings (`use_tabular = false`). Therefore the headline result in this repository is not that metadata fusion improved performance, but that the frozen visual representation alone slightly outperformed metadata fusion on the saved grouped test split.

3. Medium-severity corrosion is easier than the extremes. From the saved confusion matrix, overall three-level severity accuracy is 68.6%. Class recall is 0.550 for low, 0.897 for medium, and 0.484 for high. The model is therefore best at coarse medium-range discrimination and weaker at both ends.

8.3 Weak, negative, and inconclusive results

Weak result: late-stage severe corrosion is underestimated. The saved test predictions show a clear late-week deterioration:

- for weeks ≤ 10 : MAE 2.220, bias +0.854;
- for weeks 12–20: MAE 6.617, bias -1.314 ;
- for weeks ≥ 23 : MAE 12.075, bias -6.912 .

This pattern is visible in Figure 3 and in the D05 specimen trajectory in Figure 5. The model becomes increasingly conservative as corrosion severity grows.

Weak result: performance is group-dependent. Group metrics computed on the test subset show substantial heterogeneity:

- best treatment MAE: SA_VF, 3.62;
- worst treatment MAE: N0, 8.18;
- best series MAE: S, 4.08;
- worst series MAE: D, 13.89;
- worst specimen MAE: D05, 13.89.

This means the single headline MAE hides materially different behavior across specimen families.

Negative result: the multimodal model did not win on the held-out grouped test split. Although the multimodal branch reached a better best validation MAE (4.0161 versus 4.9595), its test MAE was worse (5.7476 versus 5.6255). This prevents any supported claim that metadata fusion improved robustness in `main_2`.

Negative result: offline predictions include physically impossible negatives. The saved prediction CSV contains 117 negative predictions in train, 7 in validation, and 13 in test, with a test minimum of -3.06% . Since the API path clips to $[0, 100]$ but the offline CSV does not, the repository currently has inconsistent output semantics.

Inconclusive result: relationship to structural damage or residual capacity. No structural label is modeled in `main_2`, so the corrosion-regression results cannot be promoted into claims about internal wire loss, ultimate load, or hidden degradation progression. The repository provides no output that directly tests those quantities here.

8.4 What the results do *not* justify claiming

The saved outputs do not justify claiming that `main_2`:

- predicts RUL or failure time;
- predicts internal corrosion mechanisms from images;
- robustly generalizes across unseen campaigns or treatments;
- reproduces the thesis RGB-threshold workflow;
- demonstrates that multimodal fusion is superior;
- provides a clean, unbiased final test estimate, because test data were also used for model choice.

9 What the model can and cannot learn

9.1 What is directly visible and therefore learnable

From the images alone, the model can plausibly learn:

- rust color and stain intensity;
- the spatial spread of visible corrosion evidence;
- coarse texture differences associated with clean versus corroded regions;
- specimen-level appearance patterns that correlate with treatment family or ageing stage.

Because the target itself is `B_Peak_Rust_Percentage_[%]`, which is a same-surface visible corrosion label, this is a relatively favorable target for an image model. The task is narrower than predicting hidden internal damage.

9.2 What may be indirectly inferable

Through either image appearance or metadata, the pipeline may indirectly infer:

- approximate exposure stage, since later weeks often exhibit greater visible corrosion;
- treatment family, either from the metadata branch or from systematic visual differences;
- campaign family, via `series` in the multimodal branch or via shared appearance characteristics.

However, such indirect inference is scientifically weaker than direct mechanistic estimation. It may reflect dataset structure rather than transferable physical understanding.

9.3 What is hidden and likely not reliably recoverable here

The current `main_2` dataset and code do not support reliable recovery of:

- internal wire loss at each week;
- ultimate load at each week;
- failure probability or time-to-threshold;
- causal degradation dynamics under new treatment schedules;

- robust extrapolation beyond the observed image-preprocessing regime.

These quantities are either hidden from the surface image, sparsely labeled only at specimen endpoints, or not represented at all in the executable training task.

9.4 Why metadata can help but also mislead

Metadata such as week, series, treatment, saline condition, and cover thickness can encode real mechanistic information. In a small dataset, they can also act as shortcuts by partially revealing expected corrosion level. The multimodal model’s lower validation MAE but worse test MAE is consistent with this trade-off: metadata likely carries signal, but the repository does not show that it generalizes more robustly than image-only prediction.

9.5 Scientific implication

Scientifically, `main_2` should be interpreted as a visible-corrosion quantification tool, not as a full structural health monitoring solution. It may be useful for automated scoring of current surface rust severity, but that is narrower than predicting hidden damage or remaining service life.

10 Traceability to the original thesis and documentation

10.1 Documented method versus executable repository

The local thesis PDF and conference paper describe a workflow centered on manual or semi-automated image preprocessing and rule-based corrosion extraction:

- the thesis describes GIMP cropping, white-balance correction, BIMP batch preprocessing, and MATLAB RGB-threshold masking for automated corrosion analysis;
- the conference paper describes a MATLAB RGB-based classification workflow and a ResNet-50 classifier trained on corrosion classes derived from that process.

By contrast, `main_2` implements a different computational path: frozen ResNet-18 embeddings, an MLP regressor, and a continuous peak-rust target.

Original documentation	What <code>main_2</code> actually reproduces	Expected consequence of the mismatch
GIMP/BIMP preprocessing with cropping, white balance, and fixed export settings (thesis)	Not reproduced in executable code. <code>main_2</code> relies on existing PNGs and torchvision transforms only.	Color-sensitive corrosion cues may shift if the current PNG set was not produced under the same preprocessing rules.
MATLAB RGB-threshold masking and binary segmentation (thesis/conference)	Not implemented in <code>main_2</code> . No masks, no RGB thresholds, no strip counting.	Feature semantics differ substantially; <code>main_2</code> cannot be claimed as a code reproduction of that pipeline.
Strip-based quantification along specimen length (conference)	Not implemented in <code>main_2</code> .	Localized corrosion structure used in the documented method is absent from the executable feature set.
ResNet-50 image classification of corrosion classes (conference)	Not reproduced. <code>main_2</code> uses frozen ResNet-18 embeddings plus regression.	Reported results are not directly comparable to conference classification metrics.
Image-based corrosion categorization into three levels (conference)	Only approximated post hoc by thresholding continuous regression outputs at 1% and 20%.	Severity confusion in <code>main_2</code> is a diagnostic convenience, not the native supervised task.
792-image thesis dataset	Partially reproduced through saved artifacts, but the current raw workbook now exposes only 791 labeled rows; one corrupted image row remains only in the saved outputs and image directory.	Fresh reruns and exact traceability to the original data snapshot are currently broken.

Table 10: Repository-grounded comparison between the documented project method and the executable `main_2` code.

10.2 What could not be recovered

The repository does not contain the original GIMP/BIMP configuration files, the MATLAB code used for RGB-threshold corrosion masking, or a documented mapping from the current raw workbook to the `NaCl%` feature expected by `main_2`. Therefore the exact preprocessing and feature-generation steps described in the documentation cannot be fully reconstructed from the current repository state.

10.3 How the report handles these mismatches

Whenever documentation claims are broader than the code, this report defers to the code and saved artifacts. Where the code itself can no longer be rerun from the current raw workbook, the report states that ambiguity explicitly rather than inferring undocumented repair steps.

11 Limitations

The main limitations of `main_2` are not generic; they are specific to the repository evidence:

1. **Small specimen count.** The saved run uses 48 specimens, even though the row count is 792. Grouped generalization is therefore estimated from only 10 test specimens.
2. **Imbalanced corrosion regime.** Stored category counts are dominated by low-rust observations, especially for the surface-rust category.

3. **Late-stage underprediction.** Severe late-week corrosion is systematically underestimated, especially for D-series specimens.
4. **No structural supervision in the executable task.** Structural endpoint labels exist but are sparse, specimen-terminal, and unused.
5. **No direct RUL supervision.** Neither the data interface nor the code contains time-to-failure labels or threshold-crossing logic.
6. **Current reproducibility drift.** The present workbook schema no longer matches the loader; the present API metadata points to another machine; the saved run therefore cannot be reproduced end to end without repair.
7. **Potential metadata confounding.** The multimodal branch includes group identity and exposure-stage features that can encode shortcuts.
8. **Test-set model selection.** The code uses the test split for model choice, so the final test metric is not a clean untouched estimate.
9. **Output-range inconsistency.** Offline saved predictions are unclipped and can be negative, while API outputs are clipped to $[0, 100]$.
10. **Limited external validity.** Because image alignment and thesis-style preprocessing are not reproduced in code, transfer to differently prepared image sets is uncertain.

12 Future work

The next steps that logically follow from the current `main_2` evidence are technical repair and scope clarification, not generic model escalation:

1. **Repair the data contract.** Update the loader to handle the current two-header workbook, parse row identifiers from `Sample Name`, and make the provenance of `NaCl%` explicit.
2. **Create an immutable manifest.** Freeze a CSV manifest that maps every PNG to its label row, including a documented decision for the corrupted image `E01-20240508-17W`.
3. **Make artifact paths relative or relocatable.** Replace absolute path storage in `run_info.json` so that inference and API deployment work on a different machine.
4. **Separate model selection from test reporting.** Choose the final branch on validation only, then retrain or refit according to a documented protocol before final testing.
5. **Constrain regression outputs.** Use bounded output heads or consistent clipping so that offline and deployed semantics agree.
6. **Add stricter evaluation.** If robust transfer matters, add leave-one-series-out, leave-one-treatment-out, or campaign-forward splits instead of only grouped random splitting.
7. **If the goal is thesis reproducibility, reintroduce the documented preprocessing.** Port or recover the GIMP/BIMP/MATLAB workflow, then compare the RGB-threshold features against the current deep-embedding approach.
8. **If the goal is structural degradation or RUL, build a separate label strategy first.** Current `main_2` does not contain the supervision needed for that objective.

A Meeting-defense questions and concise evidence-based answers

Why not direct RUL? Because `main_2` never defines an RUL target, never fits threshold-crossing time, and never trains a time-to-event model. The only supervised target is current `B_Peak_Rust_Percentage_[%]`.

Why are grouped specimen splits necessary? Each specimen is imaged repeatedly across weeks. Random row-wise splitting would place closely related images of the same physical specimen into both train and test, which would inflate performance.

Why did the multimodal model not become the final artifact? The saved run shows lower validation MAE for the multimodal branch but lower *test* MAE for the image-only branch. The code selects the model with lower test MAE, so the saved artifact is image-only.

Why does robustness appear to drop at higher corrosion levels? Late-stage test rows have much larger error and a strongly negative bias. High-severity recall in the three-level confusion matrix is only 0.484, and specimen D05 is a concrete example of severe underprediction.

Why can visible corrosion be predicted better than structural degradation? The target in `main_2` is itself a visible surface label, so the image contains direct evidence for it. Structural variables are hidden, much sparser, and not supervised in the executable task.

How much of the signal comes from metadata? The repository does not provide a clean variance decomposition. What is observable is that metadata was strong enough to improve validation MAE in the multimodal branch, but not strong enough to beat image-only on the saved grouped test split.

What are the highest-risk assumptions in the current repository state? The largest risks are: the workbook-schema assumption in the loader, the use of stale absolute artifact paths, the use of the test set for model selection, and the implicit assumption that the current PNGs still reflect the documented thesis preprocessing.

Could preprocessing mismatch explain part of the failure? Yes. The documentation emphasizes cropping and white balance before corrosion analysis, but `main_2` does not execute those steps. If the current PNG set differs from the documented preprocessing regime, the learned visual mapping can shift.

Which features are most interpretable? The tabular metadata are more interpretable than the 512-dimensional deep image embedding, but the selected saved artifact does not use them. The post-hoc low/medium/high severity bins are interpretable, but they are derived summaries rather than native model outputs.

What exactly do the reported metrics mean? MAE is the mean absolute error in percentage points of peak rust. RMSE penalizes larger errors more heavily. R^2 measures variance explained on the saved grouped test split. The three-level confusion matrix is a secondary diagnostic produced by thresholding the regression output at 1% and 20%.

Can the API be used as-is today? Not without repair. The local artifacts exist, but `run_info.json` points to absolute paths on another machine, so artifact loading fails in the current repository state.